Principles of Programming Languages 2026
Assignment 3

Responsible TA: Ariel Grunfeld

Submission Date: 25/5/2026

Part 0: Preliminaries

Structure of a TypeScript Project

Every TypeScript assignment will come with two very important files:

- `package.json` - lists the dependencies of the project.
- `tsconfig.json` - specifies the TypeScript compiler options.

Before starting to work on your assignment, open a command prompt in your assignment folder and run `npm install` to install the dependencies.

What happens when you run `npm install` and the file `package.json` is present in the folder is the following:

1. `npm` will download all required modules and their dependencies from the internet into the folder `node_modules`.
2. A file `package-lock.json` is created which lists the exact version of all the packages that have been installed.

What `tsconfig.json` controls is the way the TypeScript compiler (`tsc`) analyzes and typechecks the code in this project. We will use for all the assignments the strongest form of type-checking, which is called the "strict" mode of the `tsc` compiler.

Do not delete or change these files (*e.g.*, install new packages or change compiler options), as we will run your code against our own copy of those files, exactly the way we provide them.

If you change these files, your code may run on your machine but not when we test it, which may lead to a situation where you believe your code is correct, but you would fail to pass compilation when we grade the assignment (which means a grade of zero).

Testing Your Code

Every TypeScript assignment will have Jest as a global dependency for testing purposes (so no need to import it). In order to run the tests, save your tests in the `test` directory in a file ending with `.test.ts` and run `npm test` from a command prompt. This will activate the execution of the tests you have specified in the test file and report the results of the tests in a very nice format.

An example test file `assignmentX.test.ts` might look like this:

```
import { sum } from "../src/assignmentX";

describe("Assignment X", () => {
  it("sums two numbers", () => {
    expect(sum(1, 2)).toEqual(3);
  });
});
```

Every function you want to test must be `export`-ed, for example, in `assignmentX.ts`, so that it can be `import`-ed in the `.test.ts` file (and by our automatic test script when we grade the assignment).

```
export const sum = (a: number, b: number) => a + b;
```

You are given some basic tests in the `test` directory, just to make sure you are on the right track during the assignment.

What to Submit

You should submit a zip file called `<id1>_<id2>.zip` which has the following structure:

```
/
├── Part1.pdf
└── src/
    └── L5/
        ├── L5-typecheck.ts
        ├── L5-substitution-adt.ts
        └── L5-type-equations.ts
```

Make sure that when you extract the zip (using `unzip` on Linux), the result is flat, *i.e.*, not inside a folder (the file `Part1.pdf` is in the root directory). This structure is crucial for us to be able to import your code to our tests. Also, make sure the file is a `.zip` file – not a RAR or TAR or any other compression format.

Part 1: Theoretical Questions [35 pts]

Submit the solution to this part as `Part1.pdf`. We can't stress this enough: the file *has to be a PDF file*.

Question 1.1: [6 points]

For each of the following typing statement, write whether it is true or false. If false, explain why:

1. `{f:(T1 -> T2), g:(T1 -> T2), a: T1}` ⊢ `(f (g a)):T2`
2. `{f:(T1 * T2 -> T3)}` ⊢ `(lambda (x) (f x y)): (T2 -> T3)`
3. `{f:(T1 * T2 -> T3), y: T2}` ⊢ `(lambda (x) (f x y)):(T1 -> T3)`

Question 1.2: [15 points]

Perform type inference manually on the following expressions, using the Type Equations method.

In your answer, provide the following:

i. List the pool of fresh type variables assigned to each sub-expression.

ii. List the equations generated by rules for the sub-expressions, on the first stage of the inference algorithm. (there is no need to list the equations generated during the unification)

iii. According to the success or failure of the inference:

If the inference succeeds: Write the final substitution generated by the unification algorithm.

If the inference fails: Explain which error you get and what triggered it. (For example: For `((lambda (f x) (f x)) 4 sqrt)` it fails because the algorithm is trying to solve the conflicting equation `[Tx -> T2] = Number`)

1. `(lambda (f1 g1 x1 y1) (f1 (g1 y1 x1) (g1 x1 y1)))`
2. `((lambda (f1) (lambda (x1) (f1 x1 x1))) (lambda (y1) y1))`
3. `((lambda (f1) (lambda (x1) ((f1 x1) x1))) (lambda (y1) y1))`

Question 1.3: Sum Types [14 points]

In TypeScript, we learned how to combine union types and string literals to a pattern called *tagged union*, allowing you to discriminately pack expressions of different types under the same type. In this question, we consider the introduction of a simplified version of it to Scheme, called *sum types*.

Given two types `TE1` and `TE2`, the sum type of `TE1` and `TE2`, denoted as `(TE1 + TE2)`, has the following values:

```
(left E)   : (TE1 + TE2)   where E : TE1
(right E)  : (TE1 + TE2)   where E : TE2
```

Where `left` and `right` are added to the set of keywords of the language, and `(left E)` and `(right E)` are added to the syntax as special forms.

So, for example, both `(left 2)` and `(right #t)` are values of type `(number + boolean)`.

Semantically, the value of `(left E)` is `(left V)`, and the value of `(right E)` is `(right V)`, where `V` is the value of `E`. So, for example, the value of `(left (+ 3 4))` is `(left 7)`, and the value of `(right (and #t #f))` is `(right #f)`. (Intuitively, you can think of `(left 7)` as a symbol `'(left . 7)`, and `(right #f)` as a symbol `'(right . #f)`)

Following this logic, we get the following type constraints for the `left` and `right` special forms:

i. For the pool:

`T0`	`(left E)`
`T1`	`E`

we get the equation:

`T0 = T1 + T`

where `T` is a **fresh** type variable.

That's the equational way of saying: "Whenever `E` is a term of type `T1`, `(left E)` is a term of type `T1 + T`, for some arbitrary type `T`".

ii. For the pool:

`T0`	`(right E)`
`T1`	`E`

we get the equation:

`T0 = T + T1`

where `T` is a **fresh** type variable.

That's the equational way of saying: "Whenever `E` is a term of type `T1`, `(right E)` is a term of type `T + T1`, for some arbitrary type `T`".

Using the type constraints specified above, perform type inference manually on the following expression:

```
(lambda (f) (f (right (lambda (s) (f (left s))))))
```

In your answer, provide the following:

i. List the pool of fresh type variables assigned to each sub-expression.

ii. List the equations generated by rules for the sub-expressions, on the first stage of the inference algorithm. (there is no need to list the equations generated during the unification)

iii. According to the success or failure of the inference:

If the inference succeeds: Write the final substitution generated by the unification algorithm.

If the inference fails: Explain which error you get and what triggered it.

Part 2: Type Checking: Support `define` and `L5` Expressions [30 points]

Question 2.1: [10 points] `define`

In L5, add support for `define` expressions in the type checker. For example, the following code should be typed with `void`:

```
(define (myvar : number) 5)
```

The following code should raise a type error:

```
(define (myvar : number) (lambda (x y) (+ x y)))
```

Guidelines

Implement the function `typeofDefine` in `src/L5/L5-typecheck.ts`.

You may add other helper functions to `src/L5/L5-typecheck.ts` if you need.

Question 2.2: [20 points] `L5` Program

Add support for checking the type of an entire program, which is a sequence of expressions wrapped in a boundary:

```
(L5 <exp>+)
```

Recall that the value of the program is the value of the last expression, so similarly, the type of the program is the type of the last expression.

For example, the following code should be typed with `number`:

```
(L5 (define x 5) (and #t #f) (+ x 9))
```

Guidelines

Implement the function `typeofProgram` in `src/L5/L5-typecheck.ts`.

You may add other helper functions to `src/L5/L5-typecheck.ts` if you need.

Part 3: Type Inference: Support the `(list T)` Compound Type [35 points]

Add support for the type of (homogeneous) lists in the type inference code.

The type of lists is defined in `src/L5/TExp.ts` as `ListTExp`.

Values of type `(list T)` are either:

i. An empty list `'()`

ii. A non-empty list: A pair `'(<head> . <tail>)` where `<head>` is a value of type `T` and `<tail>` is a value of type `(list T)`

Guidelines

Make the following changes to the code:

1. [6 points] Primitive operators: In `src/L5/L5-typecheck.ts`: Modify `typeofPrim` to compute the types of `cons`, `car`, and `cdr`. They should have the following types (for an **arbitrary** type variable `T`):

```
cons   :   (T * (list T) -> (list T))
car    :   ((list T) -> T)
cdr    :   ((list T) -> (list T))
```

2. Type substitution: In `src/L5/L5-substitution-adt.ts`:
   (a) [4 points] Modify `checkNoOccurrence` to check if there are no occurrences of a type variable in a `ListTExp`.
   (b) [4 points] Modify `applySub` to apply a type substitution to a `ListTExp`.
3. Unification: In `src/L5/L5-type-equations.ts`:
   (a) [5 points] Modify `expToPool` to generate a pool of fresh type variables from an empty list value and from a non-empty list value.
   (b) [10 points] Modify `makeEquationsFromExp` to generate the appropriate equations to handle empty list values and non-empty list values. Note that for a non-empty list value, the equations have to ensure that the list is homogeneous: The head of the list must have the same type as the argument of the tail of the list.
   (c) [3 point] Modify `canUnify` to check if you can unify two list types.
   (d) [3 point] Modify `splitEquation` to split a valid equation between list types into sub-equations.

Good Luck and Have Fun!